

Disjoint Set Union

September 8, 2026

A (ID 2200110)

B (ID 2200110)

C (ID 2200110)

Department of Computer Science and Engineering

Outline

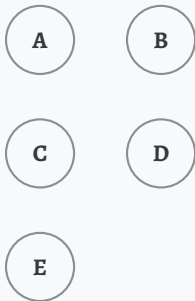
Introduction

Naive Implementation

Path Compression

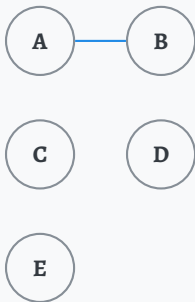
Introduction

A Familiar Problem



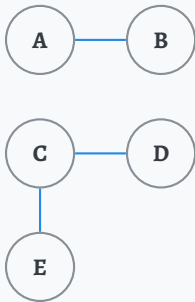
- Suppose there are five students in a classroom.
- At first, none of them know one another.

A Familiar Problem



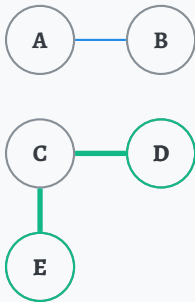
- Suppose there are five students in a classroom.
- At first, none of them know one another.
- A and B become friends.

A Familiar Problem



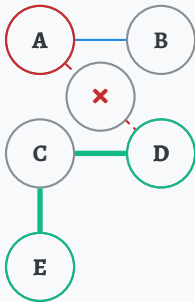
- Suppose there are five students in a classroom.
- At first, none of them know one another.
- A and B become friends.
- Separately, C become friends with D and E.

A Familiar Problem



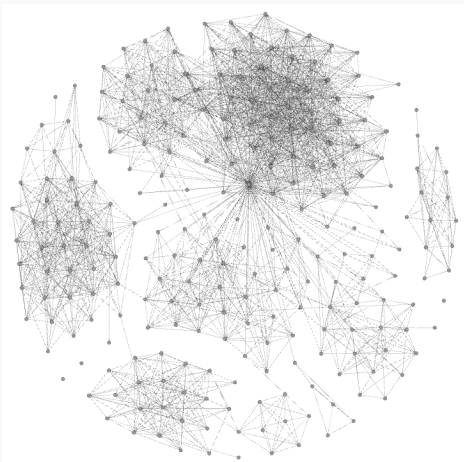
- Suppose there are five students in a classroom.
- At first, none of them know one another.
- A and B become friends.
- Separately, C become friends with D and E.
- **Are D and E friends?** Yes — indirectly, through C, they belong to same friend circle.

A Familiar Problem



- Suppose there are five students in a classroom.
- At first, none of them know one another.
- A and B become friends.
- Separately, C become friends with D and E.
- **Are D and E friends?** Yes — indirectly, through C, they belong to same friend circle.
- What about **A and D**? No — they belong to two different circles entirely.

Larger Networks



- Consider a large social network, or equally, a large computer network, with countless members.

Figure: *Social Media Network Visualization^a*

^aPinterest: Visualise Your Facebook Friend Network

Larger Networks

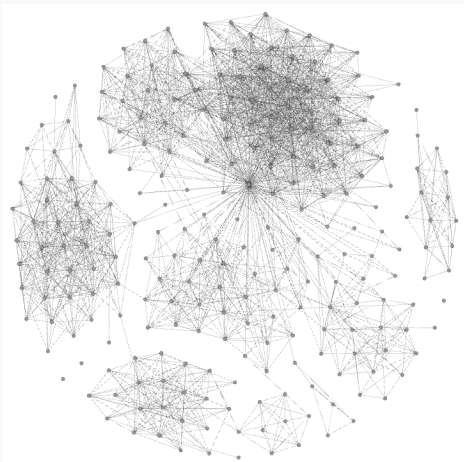


Figure: *Social Media Network Visualization*^a

- Consider a large social network, or equally, a large computer network, with countless members.
- We need to answer **efficiently**: are any two given members connected?

^aPinterest: Visualise Your Facebook Friend Network

Larger Networks

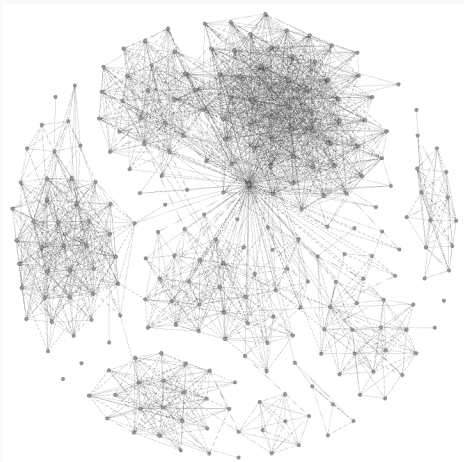


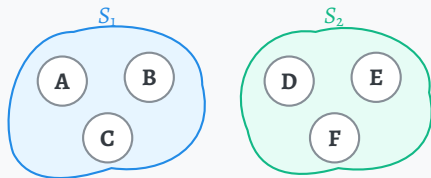
Figure: Social Media Network Visualization^a

- Consider a large social network, or equally, a large computer network, with countless members.
- We need to answer **efficiently**: are any two given members connected?

This is exactly what *Disjoint Set Union* answers.

^aPinterest: Visualise Your Facebook Friend Network

What Is a Disjoint Set Union?



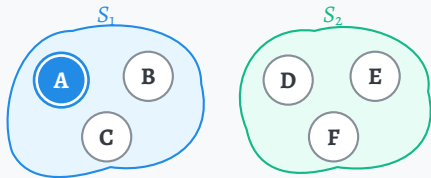
Definition

- Sets are **disjoint** if they share no elements:

$$S_i \cap S_j = \emptyset \quad \text{for } i \neq j.$$

- **Disjoint Set Union** or **DSU** are a collection of such disjoint groups.

What Is a Disjoint Set Union?



Here,

- A is the representative of S_1

Definition

- Sets are **disjoint** if they share no elements:

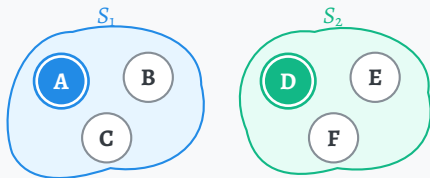
$$S_i \cap S_j = \emptyset \quad \text{for } i \neq j.$$

- **Disjoint Set Union** or **DSU** are a collection of such disjoint groups.

Representative (“leader”)

- One chosen element identifies the entire set.

What Is a Disjoint Set Union?



Here,

- A is the representative of S_1
- D is the representative of S_2

Definition

- Sets are **disjoint** if they share no elements:

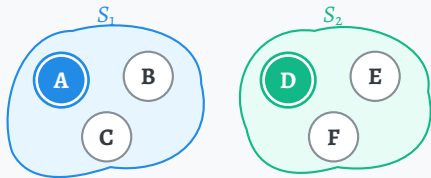
$$S_i \cap S_j = \emptyset \quad \text{for } i \neq j.$$

- **Disjoint Set Union** or **DSU** are a collection of such disjoint groups.

Representative (“leader”)

- One chosen element identifies the entire set.

What Is a Disjoint Set Union?



Here,

- A is the representative of S_1
- D is the representative of S_2

Definition

- Sets are **disjoint** if they share no elements:

$$S_i \cap S_j = \emptyset \quad \text{for } i \neq j.$$

- **Disjoint Set Union** or **DSU** are a collection of such disjoint groups.

Representative (“leader”)

- One chosen element identifies the entire set.

Same representative $\iff$ same set.

The Interface

1. `make_set(x)`
 - Creates a new set whose only member is `x`.

The Interface

1. `make_set(x)`
 - Creates a new set whose only member is `x`.
2. `find(x)`
 - Returns the representative of the set containing `x`.

The Interface

1. `make_set(x)`
 - Creates a new set whose only member is `x`.
2. `find(x)`
 - Returns the representative of the set containing `x`.
3. `union(x, y)`
 - Merges the sets containing `x` and `y` into one.

The Interface

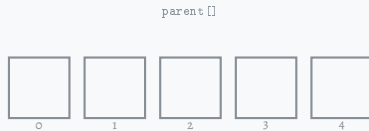
1. `make_set(x)`
 - Creates a new set whose only member is `x`.
2. `find(x)`
 - Returns the representative of the set containing `x`.
3. `union(x, y)`
 - Merges the sets containing `x` and `y` into one.

Because of the two main operations, `union` and `find`, this structure is also called **Union-Find**.

Naive Implementation

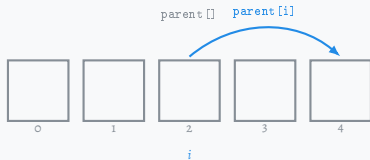
The Setup

- Maintain an array `parent[]`



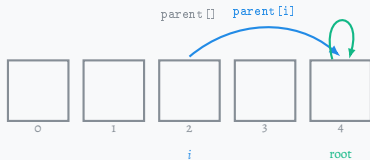
The Setup

- Maintain an array `parent[]`
- `parent[i]` stores the index of element *i*'s **immediate parent** — not necessarily its representative.



The Setup

- Maintain an array `parent[]`
- `parent[i]` stores the index of element *i*'s **immediate parent** — not necessarily its representative.
- Following parents eventually reaches the **root**, which is exactly the representative: a root points to itself, `parent[root] = root`.



make_set

Pseudocode

```
1  make_set(node):  
2      parent[node] ← node
```

Call once for each new element.

make_set

Pseudocode

```
1 make_set(node):  
2   parent[node] ← node
```

Call once for each new element.

Parent array

index	0	1	2	3	4	5	6
parent	–	–	–	–	–	–	–

– means not initialized yet.

DSU Forest

No sets created yet.

make_set

Pseudocode

```
1 make_set(node):  
2   parent[node] ← node
```

Call once for each new element.

Operations

`make_set(0): parent[0] = 0`

Parent array

index	0	1	2	3	4	5	6
parent	0	–	–	–	–	–	–

– means not initialized yet.

DSU Forest



make_set

Pseudocode

```
1 make_set(node):  
2   parent[node] ← node
```

Call once for each new element.

Operations

```
make_set(0): parent[0] = 0  
make_set(1): parent[1] = 1
```

Parent array

index	0	1	2	3	4	5	6
parent	0	1	-	-	-	-	-

- means not initialized yet.

DSU Forest



make_set

Pseudocode

```
1  make_set(node):  
2      parent[node] ← node
```

Call once for each new element.

Operations

```
make_set(0): parent[0] = 0  
make_set(1): parent[1] = 1  
make_set(2): parent[2] = 2
```

Parent array

index	0	1	2	3	4	5	6
parent	0	1	2	–	–	–	–

– means not initialized yet.

DSU Forest



make_set

Pseudocode

```
1 make_set(node):  
2     parent[node] ← node
```

Call once for each new element.

Operations

```
make_set(0): parent[0] = 0  
make_set(1): parent[1] = 1  
make_set(2): parent[2] = 2  
make_set(3): parent[3] = 3
```

Parent array

index	0	1	2	3	4	5	6
parent	0	1	2	3	–	–	–

– means not initialized yet.

DSU Forest



make_set

Pseudocode

```
1 make_set(node):  
2     parent[node] ← node
```

Call once for each new element.

Operations

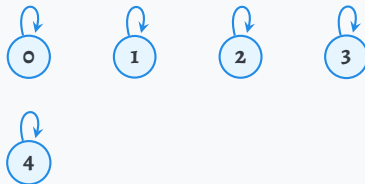
```
make_set(0): parent[0] = 0  
make_set(1): parent[1] = 1  
make_set(2): parent[2] = 2  
make_set(3): parent[3] = 3  
make_set(4): parent[4] = 4
```

Parent array

index	0	1	2	3	4	5	6
parent	0	1	2	3	4	–	–

– means not initialized yet.

DSU Forest



make_set

Pseudocode

```
1 make_set(node):  
2   parent[node] ← node
```

Call once for each new element.

Operations

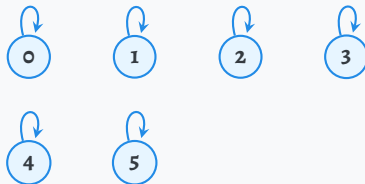
```
make_set(0): parent[0] = 0  
make_set(1): parent[1] = 1  
make_set(2): parent[2] = 2  
make_set(3): parent[3] = 3  
make_set(4): parent[4] = 4  
make_set(5): parent[5] = 5
```

Parent array

index	0	1	2	3	4	5	6
parent	0	1	2	3	4	5	–

– means not initialized yet.

DSU Forest



make_set

Pseudocode

```
1 make_set(node):  
2   parent[node] ← node
```

Call once for each new element.

Operations

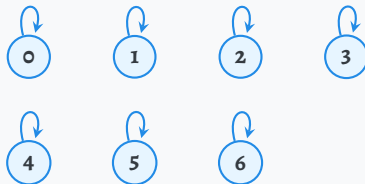
```
make_set(0): parent[0] = 0  
make_set(1): parent[1] = 1  
make_set(2): parent[2] = 2  
make_set(3): parent[3] = 3  
make_set(4): parent[4] = 4  
make_set(5): parent[5] = 5  
make_set(6): parent[6] = 6
```

Parent array

index	0	1	2	3	4	5	6
parent	0	1	2	3	4	5	6

– means not initialized yet.

DSU Forest



find: Follow Parents to the Root

Pseudocode

```
1 find(node):  
2     if parent[node] = node then  
3         return node  
4     return find(parent[node])
```

find: Follow Parents to the Root

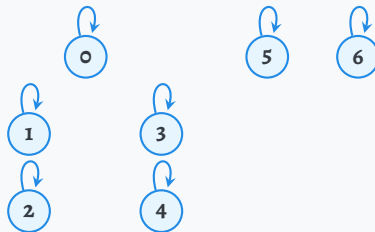
Pseudocode

```
1 find(node):  
2     if parent[node] = node then  
3         return node  
4     return find(parent[node])
```

Parent array

index	0	1	2	3	4	5	6
parent	0	1	2	3	4	5	6

DSU Forest



find: Follow Parents to the Root

Pseudocode

```
1 find(node):  
2     if parent[node] = node then  
3         return node  
4     return find(parent[node])
```

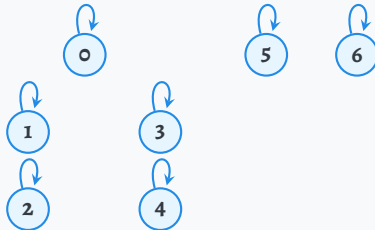
Operations

find(4)

Parent array

index	0	1	2	3	4	5	6
parent	0	1	2	3	4	5	6

DSU Forest



find: Follow Parents to the Root

Pseudocode

```
1 find(node):  
2     if parent[node] = node then  
3         return node  
4     return find(parent[node])
```

Operations

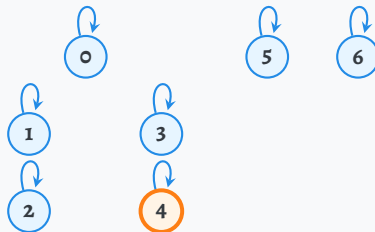
find(4)

- Read `parent[4] = 4`.

Parent array

index	0	1	2	3	4	5	6
parent	0	1	2	3	4	5	6

DSU Forest



find: Follow Parents to the Root

Pseudocode

```
1 find(node):  
2     if parent[node] = node then  
3         return node  
4     return find(parent[node])
```

Operations

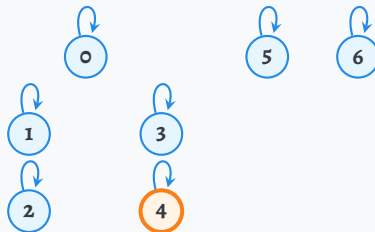
find(4)

- Read `parent[4] = 4`.
- The node is its own parent: return 4.

Parent array

index	0	1	2	3	4	5	6
parent	0	1	2	3	4	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1  union(first, second):  
2      rootA ← find(first)  
3      rootB ← find(second)  
4      if rootA ≠ rootB then  
5          parent[rootB] ← rootA
```

union: Find Both Roots, Then Link

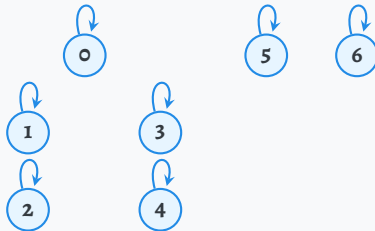
Pseudocode

```
1 union(first, second):  
2   rootA ← find(first)  
3   rootB ← find(second)  
4   if rootA ≠ rootB then  
5     parent[rootB] ← rootA
```

Parent array

index	0	1	2	3	4	5	6
parent	0	1	2	3	4	5	6

DSU Forest



Pseudocode

```

1  union(first, second):
2      rootA ← find(first)
3      rootB ← find(second)
4      if rootA ≠ rootB then
5          parent[rootB] ← rootA

```

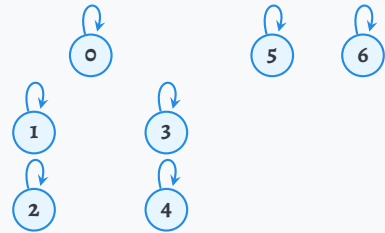
Operations

```
union(1, 2)
```

Parent array

index	0	1	2	3	4	5	6
parent	0	1	2	3	4	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1 union(first, second):  
2     rootA ← find(first)  
3     rootB ← find(second)  
4     if rootA ≠ rootB then  
5         parent[rootB] ← rootA
```

Operations

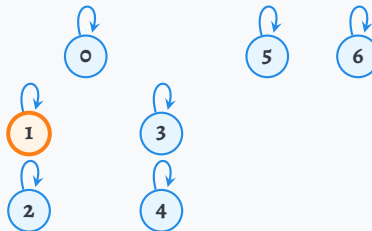
union(1, 2)

find(1): parent[1] = 1 $\Rightarrow$ rootA = 1

Parent array

index	0	1	2	3	4	5	6
parent	0	1	2	3	4	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1 union(first, second):  
2     rootA ← find(first)  
3     rootB ← find(second)  
4     if rootA ≠ rootB then  
5         parent[rootB] ← rootA
```

Operations

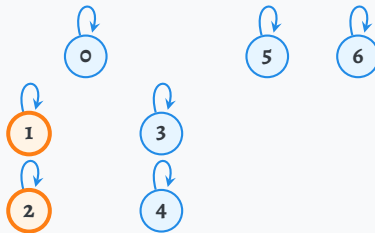
union(1, 2)

find(1): parent[1] = 1 $\Rightarrow$ rootA = 1
find(2): parent[2] = 2 $\Rightarrow$ rootB = 2

Parent array

index	0	1	2	3	4	5	6
parent	0	1	2	3	4	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1 union(first, second):  
2     rootA ← find(first)  
3     rootB ← find(second)  
4     if rootA ≠ rootB then  
5         parent[rootB] ← rootA
```

Operations

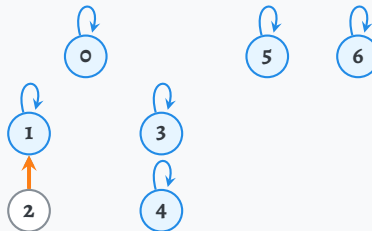
union(1, 2) parent[2] ← 1

find(1): parent[1] = 1 ⇒ rootA = 1
find(2): parent[2] = 2 ⇒ rootB = 2
Different roots: attach 2 under 1.

Parent array

index	0	1	2	3	4	5	6
parent	0	1	1	3	4	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1  union(first, second):  
2      rootA ← find(first)  
3      rootB ← find(second)  
4      if rootA ≠ rootB then  
5          parent[rootB] ← rootA
```

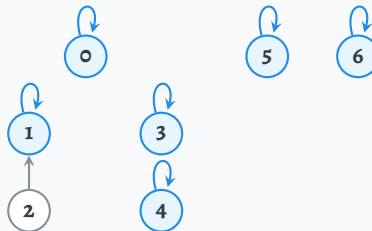
Operations

```
union(1, 2)  parent[2] ← 1  
union(0, 1)
```

Parent array

index	0	1	2	3	4	5	6
parent	0	1	1	3	4	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1 union(first, second):  
2     rootA ← find(first)  
3     rootB ← find(second)  
4     if rootA ≠ rootB then  
5         parent[rootB] ← rootA
```

Operations

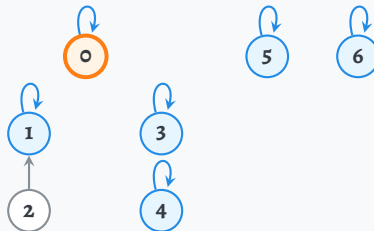
```
union(1, 2)  parent[2] ← 1  
union(0, 1)
```

```
find(0): parent[0] = 0 ⇒ rootA = 0
```

Parent array

index	0	1	2	3	4	5	6
parent	0	1	1	3	4	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1 union(first, second):  
2     rootA ← find(first)  
3     rootB ← find(second)  
4     if rootA ≠ rootB then  
5         parent[rootB] ← rootA
```

Operations

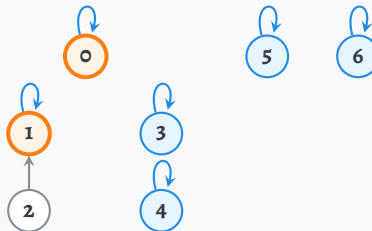
```
union(1, 2)  parent[2] ← 1  
union(0, 1)
```

```
find(0): parent[0] = 0 ⇒ rootA = 0  
find(1): parent[1] = 1 ⇒ rootB = 1
```

Parent array

index	0	1	2	3	4	5	6
parent	0	1	1	3	4	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1 union(first, second):  
2     rootA ← find(first)  
3     rootB ← find(second)  
4     if rootA ≠ rootB then  
5         parent[rootB] ← rootA
```

Operations

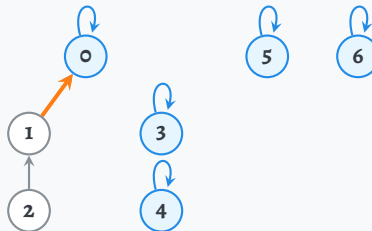
union(1, 2) parent[2] ← 1
union(0, 1) parent[1] ← 0

find(0): parent[0] = 0 ⇒ rootA = 0
find(1): parent[1] = 1 ⇒ rootB = 1
Different roots: attach 1 under 0.

Parent array

index	0	1	2	3	4	5	6
parent	0	0	1	3	4	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1  union(first, second):  
2      rootA ← find(first)  
3      rootB ← find(second)  
4      if rootA ≠ rootB then  
5          parent[rootB] ← rootA
```

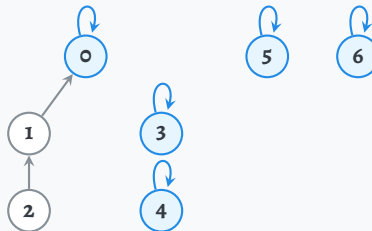
Operations

```
union(1, 2)  parent[2] ← 1  
union(0, 1)  parent[1] ← 0  
union(3, 4)
```

Parent array

index	0	1	2	3	4	5	6
parent	0	0	1	3	4	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1 union(first, second):  
2     rootA ← find(first)  
3     rootB ← find(second)  
4     if rootA ≠ rootB then  
5         parent[rootB] ← rootA
```

Operations

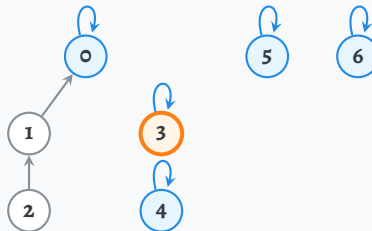
```
union(1, 2) parent[2] ← 1  
union(0, 1) parent[1] ← 0  
union(3, 4)
```

```
find(3): parent[3] = 3 ⇒ rootA = 3
```

Parent array

index	0	1	2	3	4	5	6
parent	0	0	1	3	4	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1 union(first, second):  
2     rootA ← find(first)  
3     rootB ← find(second)  
4     if rootA ≠ rootB then  
5         parent[rootB] ← rootA
```

Operations

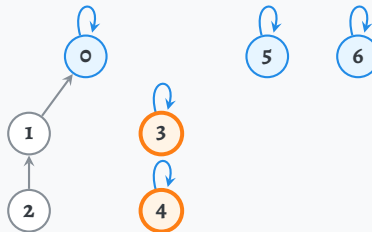
```
union(1, 2) parent[2] ← 1  
union(0, 1) parent[1] ← 0  
union(3, 4)
```

```
find(3): parent[3] = 3 ⇒ rootA = 3  
find(4): parent[4] = 4 ⇒ rootB = 4
```

Parent array

index	0	1	2	3	4	5	6
parent	0	0	1	3	4	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1 union(first, second):  
2     rootA ← find(first)  
3     rootB ← find(second)  
4     if rootA ≠ rootB then  
5         parent[rootB] ← rootA
```

Operations

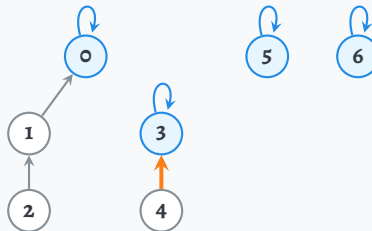
```
union(1, 2) parent[2] ← 1  
union(0, 1) parent[1] ← 0  
union(3, 4) parent[4] ← 3
```

```
find(3): parent[3] = 3 ⇒ rootA = 3  
find(4): parent[4] = 4 ⇒ rootB = 4  
Different roots: attach 4 under 3.
```

Parent array

index	0	1	2	3	4	5	6
parent	0	0	1	3	3	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1 union(first, second):  
2     rootA ← find(first)  
3     rootB ← find(second)  
4     if rootA ≠ rootB then  
5         parent[rootB] ← rootA
```

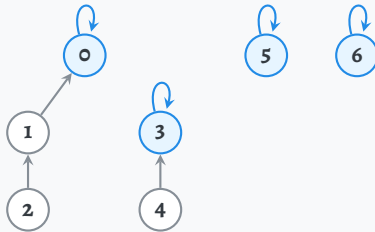
Operations

```
union(1, 2) parent[2] ← 1  
union(0, 1) parent[1] ← 0  
union(3, 4) parent[4] ← 3  
union(2, 4)
```

Parent array

index	0	1	2	3	4	5	6
parent	0	0	1	3	3	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1 union(first, second):  
2     rootA ← find(first)  
3     rootB ← find(second)  
4     if rootA ≠ rootB then  
5         parent[rootB] ← rootA
```

Operations

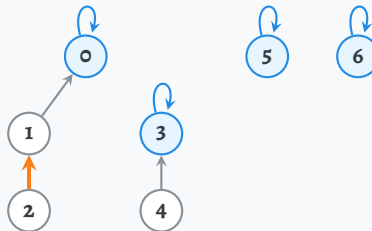
```
union(1, 2)  parent[2] ← 1  
union(0, 1)  parent[1] ← 0  
union(3, 4)  parent[4] ← 3  
union(2, 4)
```

```
find(2): 2 → 1
```

Parent array

index	0	1	2	3	4	5	6
parent	0	0	1	3	3	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1 union(first, second):  
2     rootA ← find(first)  
3     rootB ← find(second)  
4     if rootA ≠ rootB then  
5         parent[rootB] ← rootA
```

Operations

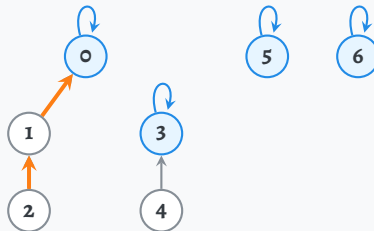
```
union(1, 2) parent[2] ← 1  
union(0, 1) parent[1] ← 0  
union(3, 4) parent[4] ← 3  
union(2, 4)
```

```
find(2): 2 → 1 → 0
```

Parent array

index	0	1	2	3	4	5	6
parent	0	0	1	3	3	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1 union(first, second):  
2     rootA ← find(first)  
3     rootB ← find(second)  
4     if rootA ≠ rootB then  
5         parent[rootB] ← rootA
```

Operations

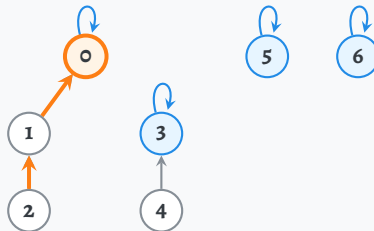
```
union(1, 2) parent[2] ← 1  
union(0, 1) parent[1] ← 0  
union(3, 4) parent[4] ← 3  
union(2, 4)
```

find(2): $2 \rightarrow 1 \rightarrow 0$; $\text{parent}[0] = 0 \Rightarrow \text{rootA} = 0$

Parent array

index	0	1	2	3	4	5	6
parent	0	0	1	3	3	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1 union(first, second):  
2     rootA ← find(first)  
3     rootB ← find(second)  
4     if rootA ≠ rootB then  
5         parent[rootB] ← rootA
```

Operations

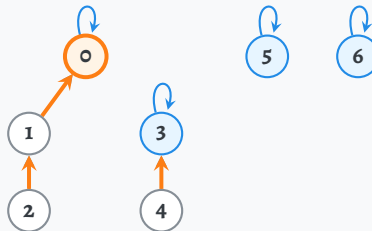
```
union(1, 2)  parent[2] ← 1  
union(0, 1)  parent[1] ← 0  
union(3, 4)  parent[4] ← 3  
union(2, 4)
```

```
find(2): 2 → 1 → 0; parent[0] = 0 ⇒ rootA = 0  
find(4): 4 → 3
```

Parent array

index	0	1	2	3	4	5	6
parent	0	0	1	3	3	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1 union(first, second):  
2     rootA ← find(first)  
3     rootB ← find(second)  
4     if rootA ≠ rootB then  
5         parent[rootB] ← rootA
```

Operations

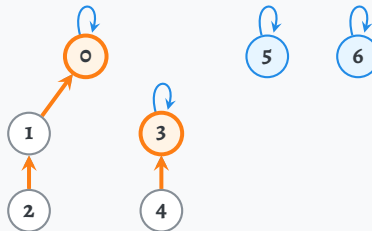
```
union(1, 2)  parent[2] ← 1  
union(0, 1)  parent[1] ← 0  
union(3, 4)  parent[4] ← 3  
union(2, 4)
```

```
find(2): 2 → 1 → 0; parent[0] = 0 ⇒ rootA = 0  
find(4): 4 → 3; parent[3] = 3 ⇒ rootB = 3
```

Parent array

index	0	1	2	3	4	5	6
parent	0	0	1	3	3	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1 union(first, second):  
2     rootA ← find(first)  
3     rootB ← find(second)  
4     if rootA ≠ rootB then  
5         parent[rootB] ← rootA
```

Operations

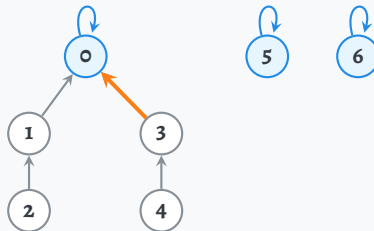
```
union(1, 2)  parent[2] ← 1  
union(0, 1)  parent[1] ← 0  
union(3, 4)  parent[4] ← 3  
union(2, 4)  parent[3] ← 0
```

find(2): $2 \rightarrow 1 \rightarrow 0$; $\text{parent}[0] = 0 \Rightarrow \text{rootA} = 0$
find(4): $4 \rightarrow 3$; $\text{parent}[3] = 3 \Rightarrow \text{rootB} = 3$
Different roots: attach 3 under 0, not 4 under 2.

Parent array

index	0	1	2	3	4	5	6
parent	0	0	1	0	3	5	6

DSU Forest



Path Compression

Naive find: The Same Work Again

Pseudocode

```
1 find(node):  
2     if parent[node] = node then  
3         return node  
4     return find(parent[node])
```

Operations

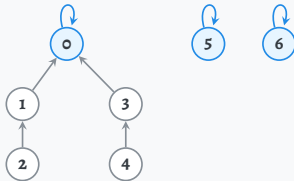
- Call find(2):

Parent array

index	0	1	2	3	4	5	6
parent	0	0	1	0	3	5	6

Exactly where the previous union left us.

DSU Forest



Naive find: The Same Work Again

Pseudocode

```
1 find(node):  
2   if parent[node] = node then  
3     return node  
4   return find(parent[node])
```

Operations

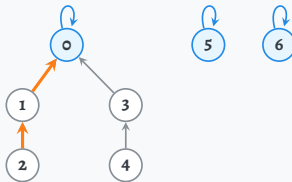
- Call $\text{find}(2)$: $2 \rightarrow 1 \rightarrow 0$.

Parent array

index	0	1	2	3	4	5	6
parent	0	0	1	0	3	5	6

Exactly where the previous union left us.

DSU Forest



Naive find: The Same Work Again

Pseudocode

```
1 find(node):  
2   if parent[node] = node then  
3     return node  
4   return find(parent[node])
```

Operations

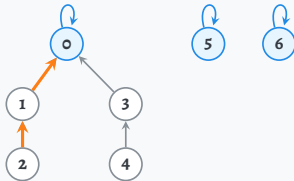
- Call `find(2)`: $2 \rightarrow 1 \rightarrow 0$.
- Call it again: **the same two links!**

Parent array

index	0	1	2	3	4	5	6
parent	0	0	1	0	3	5	6

Exactly where the previous union left us.

DSU Forest



Naive find: The Same Work Again

Pseudocode

```
1 find(node):  
2   if parent[node] = node then  
3     return node  
4   return find(parent[node])
```

Operations

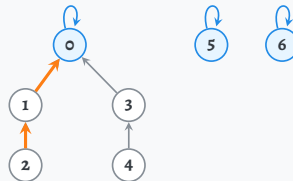
- Call `find(2)`: $2 \rightarrow 1 \rightarrow 0$.
- Call it again: **the same two links!**
- The answer is returned, but no parent is updated.

Parent array

index	0	1	2	3	4	5	6
parent	0	0	1	0	3	5	6

Exactly where the previous union left us.

DSU Forest



Naive find: The Same Work Again

Pseudocode

```
1 find(node):  
2     if parent[node] = node then  
3         return node  
4     return find(parent[node])
```

Operations

- Call `find(2)`: $2 \rightarrow 1 \rightarrow 0$.
- Call it again: **the same two links!**
- The answer is returned, but no parent is updated.

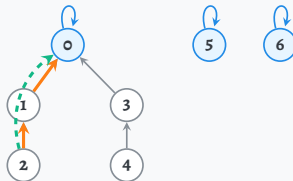
Can we remember the shortcut to 0?

Parent array

index	0	1	2	3	4	5	6
parent	0	0	1	0	3	5	6

Exactly where the previous union left us.

DSU Forest



Naive Union Can Build a Chain

A fresh worst-case example

Start again with seven singleton sets.

Our union rule: $\text{parent}[\text{rootB}] \leftarrow \text{rootA}$.

- $\text{union}(1,0), \text{union}(2,1),$
 $\text{union}(3,2), \dots, \text{union}(6,5).$

Naive Union Can Build a Chain

A fresh worst-case example

Start again with seven singleton sets.

Our union rule: $\text{parent}[\text{rootB}] \leftarrow \text{rootA}$.

- $\text{union}(1,0), \text{union}(2,1),$
 $\text{union}(3,2), \dots, \text{union}(6,5).$

Parent array

index	0	1	2	3	4	5	6
parent	1	2	3	4	5	6	6

Naive Union Can Build a Chain

A fresh worst-case example

Start again with seven singleton sets.

Our union rule: $\text{parent}[\text{rootB}] \leftarrow \text{rootA}$.

- $\text{union}(1,0), \text{union}(2,1),$
 $\text{union}(3,2), \dots, \text{union}(6,5).$

Parent array

index	0	1	2	3	4	5	6
parent	1	2	3	4	5	6	6

DSU Forest

Arrows point to parents; 6 is the root.



Naive Union Can Build a Chain

A fresh worst-case example

Start again with seven singleton sets.

Our union rule: $\text{parent}[\text{rootB}] \leftarrow \text{rootA}$.

- $\text{union}(1,0), \text{union}(2,1),$
 $\text{union}(3,2), \dots, \text{union}(6,5).$
- $\text{find}(0)$ follows **six links**.

Parent array

index	0	1	2	3	4	5	6
parent	1	2	3	4	5	6	6

DSU Forest

Arrows point to parents; 6 is the root.



Naive Union Can Build a Chain

A fresh worst-case example

Start again with seven singleton sets.

Our union rule: $\text{parent}[\text{rootB}] \leftarrow \text{rootA}$.

- $\text{union}(1,0), \text{union}(2,1),$
 $\text{union}(3,2), \dots, \text{union}(6,5)$.
- $\text{find}(0)$ follows **six links**.
- For n elements: up to $n - 1$ links, so one find can take $\Theta(n)$.

Parent array

index	0	1	2	3	4	5	6
parent	1	2	3	4	5	6	6

DSU Forest

Arrows point to parents; 6 is the root.



Naive union also calls find ,
so it can take $\Theta(n)$ too.

Path Compression: Save the Root on the Way Back

Before: only return the root

```
1 find(node):  
2     if parent[node] = node then  
3         return node  
4     return find(parent[node])
```

- Follow parents until reaching the root, exactly as before.

Path Compression: Save the Root on the Way Back

Before: only return the root

```
1 find(node):  
2     if parent[node] = node then  
3         return node  
4     return find(parent[node])
```

- Follow parents until reaching the root, exactly as before.
- As recursion returns, make **every visited node** point directly to that root.

After: also update each visited parent

```
1 find(node):  
2     if parent[node] = node then  
3         return node  
4     parent[node] ← find(parent[node])  
5     return parent[node]
```

Path Compression: Save the Root on the Way Back

Before: only return the root

```
1 find(node):  
2     if parent[node] = node then  
3         return node  
4     return find(parent[node])
```

- Follow parents until reaching the root, exactly as before.
- As recursion returns, make **every visited node** point directly to that root.
- Same set. Same representative. **Shorter paths.**

After: also update each visited parent

```
1 find(node):  
2     if parent[node] = node then  
3         return node  
4     parent[node] ← find(parent[node])  
5     return parent[node]
```


Path Compression: Save the Root on the Way Back

Before: only return the root

```
1 find(node):  
2     if parent[node] = node then  
3         return node  
4     return find(parent[node])
```

After: also update each visited parent

```
1 find(node):  
2     if parent[node] = node then  
3         return node  
4     parent[node] ← find(parent[node])  
5     return parent[node]
```

- Follow parents until reaching the root, exactly as before.
- As recursion returns, make **every visited node** point directly to that root.
- Same set. Same representative. **Shorter paths.**

Find the root, then remember it.

No change to make_set or the union pseudocode.

Simulating `find(0)`: First Reach the Root

Pseudocode

```
1 find(node):  
2   if parent[node] = node then  
3     return node  
4   parent[node] ← find(parent[node])  
5   return parent[node]
```

Operations

- Begin with the chain: call `find(0)`.

Parent array

index	0	1	2	3	4	5	6
parent	1	2	3	4	5	6	6

No assignments yet: recursive calls are still pending.

DSU Forest

Follow the original parent pointers.



Simulating `find(0)`: First Reach the Root

Pseudocode

```
1 find(node):  
2   if parent[node] = node then  
3     return node  
4   parent[node] ← find(parent[node])  
5   return parent[node]
```

Operations

- Begin with the chain: call `find(0)`.
- Calls: $0 \rightarrow 1$.

Parent array

index	0	1	2	3	4	5	6
parent	1	2	3	4	5	6	6

No assignments yet: recursive calls are still pending.

DSU Forest

Follow the original parent pointers.



Simulating `find(0)`: First Reach the Root

Pseudocode

```
1 find(node):  
2   if parent[node] = node then  
3     return node  
4   parent[node] ← find(parent[node])  
5   return parent[node]
```

Operations

- Begin with the chain: call `find(0)`.
- Calls: $0 \rightarrow 1 \rightarrow 2$.

Parent array

index	0	1	2	3	4	5	6
parent	1	2	3	4	5	6	6

No assignments yet: recursive calls are still pending.

DSU Forest

Follow the original parent pointers.



Simulating `find(0)`: First Reach the Root

Pseudocode

```
1 find(node):  
2   if parent[node] = node then  
3     return node  
4   parent[node] ← find(parent[node])  
5   return parent[node]
```

Operations

- Begin with the chain: call `find(0)`.
- Calls: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$.

Parent array

index	0	1	2	3	4	5	6
parent	1	2	3	4	5	6	6

No assignments yet: recursive calls are still pending.

DSU Forest

Follow the original parent pointers.



Simulating `find(0)`: First Reach the Root

Pseudocode

```
1 find(node):  
2   if parent[node] = node then  
3     return node  
4   parent[node] ← find(parent[node])  
5   return parent[node]
```

Operations

- Begin with the chain: call `find(0)`.
- Calls: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$.

Parent array

index	0	1	2	3	4	5	6
parent	1	2	3	4	5	6	6

No assignments yet: recursive calls are still pending.

DSU Forest

Follow the original parent pointers.



Simulating `find(0)`: First Reach the Root

Pseudocode

```
1 find(node):  
2   if parent[node] = node then  
3     return node  
4   parent[node] ← find(parent[node])  
5   return parent[node]
```

Operations

- Begin with the chain: call `find(0)`.
- Calls: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$.

Parent array

index	0	1	2	3	4	5	6
parent	1	2	3	4	5	6	6

No assignments yet: recursive calls are still pending.

DSU Forest

Follow the original parent pointers.



Simulating `find(0)`: First Reach the Root

Pseudocode

```
1 find(node):  
2   if parent[node] = node then  
3     return node  
4   parent[node] ← find(parent[node])  
5   return parent[node]
```

Operations

- Begin with the chain: call `find(0)`.
- Calls: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$.

Parent array

index	0	1	2	3	4	5	6
parent	1	2	3	4	5	6	6

No assignments yet: recursive calls are still pending.

DSU Forest

Follow the original parent pointers.



Simulating `find(0)`: First Reach the Root

Pseudocode

```
1 find(node):  
2   if parent[node] = node then  
3     return node  
4   parent[node] ← find(parent[node])  
5   return parent[node]
```

Operations

- Begin with the chain: call `find(0)`.
- Calls: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$.
- `parent[6] = 6`: return 6.

Now the suspended calls can update their parents.

Parent array

index	0	1	2	3	4	5	6
parent	1	2	3	4	5	6	6

No assignments yet: recursive calls are still pending.

DSU Forest

Root found: return 6.



Returning: Every Visited Node Gets the Shortcut

Pseudocode

```
1 find(node):  
2   if parent[node] = node then  
3     return node  
4   parent[node] ← find(parent[node])  
5   return parent[node]
```

Operations

find(6) returns 6.

Return order: 6, 5, 4, 3, 2, 1, 0.

Parent array

index	0	1	2	3	4	5	6
parent	1	2	3	4	5	6	6

Orange cell: active parent entry.

Teal arrow: a saved shortcut.

DSU Forest



Returning: Every Visited Node Gets the Shortcut

Pseudocode

```
1 find(node):  
2   if parent[node] = node then  
3     return node  
4   parent[node] ← find(parent[node])  
5   return parent[node]
```

Operations

parent[5] ← 6; return 6.

Already points to the root: no visible change.

Return order: 6, 5, 4, 3, 2, 1, 0.

Parent array

index	0	1	2	3	4	5	6
parent	1	2	3	4	5	6	6

Orange cell: active parent entry.

Teal arrow: a saved shortcut.

DSU Forest



Returning: Every Visited Node Gets the Shortcut

Pseudocode

```
1 find(node):  
2   if parent[node] = node then  
3     return node  
4   parent[node] ← find(parent[node])  
5   return parent[node]
```

Operations

parent[4] ← 6; return 6.

Replace 4 → 5 with 4 → 6.

Return order: 6, 5, 4, 3, 2, 1, 0.

Parent array

index	0	1	2	3	4	5	6
parent	1	2	3	4	6	6	6

Orange cell: active parent entry.

Teal arrow: a saved shortcut.

DSU Forest



Returning: Every Visited Node Gets the Shortcut

Pseudocode

```
1 find(node):  
2   if parent[node] = node then  
3     return node  
4   parent[node] ← find(parent[node])  
5   return parent[node]
```

Operations

parent[3] ← 6; return 6.

Replace 3 → 4 with 3 → 6.

Return order: 6, 5, 4, 3, 2, 1, 0.

Parent array

index	0	1	2	3	4	5	6
parent	1	2	3	6	6	6	6

Orange cell: active parent entry.

Teal arrow: a saved shortcut.

DSU Forest



Returning: Every Visited Node Gets the Shortcut

Pseudocode

```
1 find(node):  
2   if parent[node] = node then  
3     return node  
4   parent[node] ← find(parent[node])  
5   return parent[node]
```

Operations

parent[2] ← 6; return 6.

Replace 2 → 3 with 2 → 6.

Return order: 6, 5, 4, 3, 2, 1, 0.

Parent array

index	0	1	2	3	4	5	6
parent	1	2	6	6	6	6	6

Orange cell: active parent entry.

Teal arrow: a saved shortcut.

DSU Forest



Returning: Every Visited Node Gets the Shortcut

Pseudocode

```
1 find(node):  
2   if parent[node] = node then  
3     return node  
4   parent[node] ← find(parent[node])  
5   return parent[node]
```

Operations

parent[1] ← 6; return 6.

Replace 1 → 2 with 1 → 6.

Return order: 6, 5, 4, 3, 2, 1, 0.

Parent array

index	0	1	2	3	4	5	6
parent	1	6	6	6	6	6	6

Orange cell: active parent entry.

Teal arrow: a saved shortcut.

DSU Forest



Returning: Every Visited Node Gets the Shortcut

Pseudocode

```
1 find(node):  
2   if parent[node] = node then  
3     return node  
4   parent[node] ← find(parent[node])  
5   return parent[node]
```

Operations

parent[0] ← 6; return 6.

Done: every non-root now points to 6.

Return order: 6, 5, 4, 3, 2, 1, 0.

Parent array

index	0	1	2	3	4	5	6
parent	6	6	6	6	6	6	6

Orange cell: active parent entry.

Teal arrow: a saved shortcut.

DSU Forest



The Next `find(0)`: Six Links Become One

Before compression

0 → 1 → 2 → 3 → 4 → 5 → 6

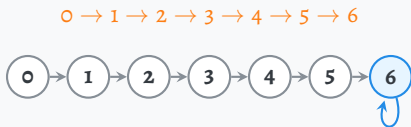


First search: six parent links.

Without compression, every repeat costs six.

The Next `find(0)`: Six Links Become One

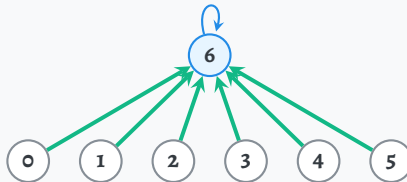
Before compression



First search: six parent links.

Without compression, every repeat costs six.

After compression: the same nodes rearranged



The Next `find(0)`: Six Links Become One

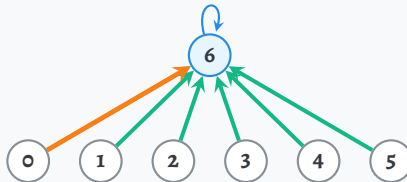
Before compression

$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$



First search: six parent links.
Without compression, every repeat costs six.

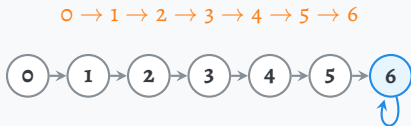
After compression: the same nodes rearranged



Next search: one parent link, $0 \rightarrow 6$.
Other visited nodes benefit too.

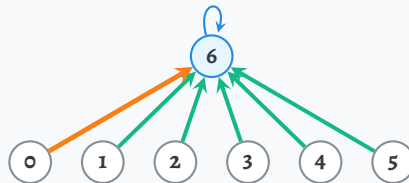
The Next `find(0)`: Six Links Become One

Before compression



First search: six parent links.
Without compression, every repeat costs six.

After compression: the same nodes rearranged



Next search: one parent link, $0 \rightarrow 6$.
Other visited nodes benefit too.

Time complexity (n elements)

Naive: $O(n)$ worst case vs. **Path compression alone:** $O(\log n)$ amortized.

The Next `find(0)`: Six Links Become One

Before compression

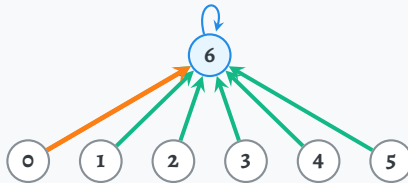
$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$



First search: six parent links.

Without compression, every repeat costs six.

After compression: the same nodes rearranged



Next search: one parent link, $0 \rightarrow 6$.

Other visited nodes benefit too.

Time complexity (n elements)

Naive: $O(n)$ worst case vs. **Path compression alone:** $O(\log n)$ amortized.

Amortized = averaged over an operation sequence;
a single `find` can still take $O(n)$.